

Implementing Collaborative Filtering with TensorFlow

You are used to TensorFlow for Neural Networks, but it is also excellent for other machine learning algorithms because of one superpower: **Auto Diff (Automatic Differentiation)**.

The Power of Auto Diff

Normally, to use Gradient Descent, you need to manually calculate the partial derivatives of your cost function (using calculus).

- **Without TensorFlow:** You must do the math to find $\frac{\partial J}{\partial w}$, $\frac{\partial J}{\partial b}$, etc., and write code for it.
 - **With TensorFlow:** You just write the code to calculate the cost J . TensorFlow automatically figures out the derivatives for you.
-

How it Works: GradientTape

TensorFlow uses a tool called `tf.GradientTape` to record the operations that happen inside a block of code.

Basic Example (Simple Regression):

We want to minimize $J = (wx - 1)^2$.

```
1 w = tf.Variable(3.0) # Initialize parameter w
2 x = 1.0
3 y = 1.0
4 alpha = 0.01        # Learning rate
5
6 for i in range(30):
7     with tf.GradientTape() as tape:
8         # 1. Compute the cost J
9         fw = w * x
10        J = (fw - y)**2
11
12        # 2. Automatically calculate the derivative dJ/dw
13        grads = tape.gradient(J, w)
14
15        # 3. Update the parameter w using gradient descent
16        w.assign_sub(alpha * grads)
```

- `w.assign_sub(...)` is TensorFlow's way of doing $w = w - \dots$
-

Implementing Collaborative Filtering

For Collaborative Filtering, the cost function is complex (involving w , b , and x). Because this custom cost function doesn't fit into a standard Neural Network layer (like `Dense`), we can't use the simple `model.compile / model.fit` syntax.

Instead, we use a **Custom Training Loop** with `GradientTape`.

The Algorithm Structure:

1. **Define Optimizer:** Use a powerful optimizer like **Adam** instead of basic gradient descent.

```
1 | optimizer = keras.optimizers.Adam(learning_rate=1e-1)
```

2. **Training Loop:**

```
1 | for iter in range(200):
2 |     with tf.GradientTape() as tape:
3 |         # Calculate the full Collaborative Filtering Cost J
4 |         # (Using w, b, x, Y_norm, R, lambda, etc.)
5 |         cost_value = cofi_cost_func_v(X, W, b, Ynorm, R, num_users,
6 |                                     num_movies, lambda_)
7 |
8 |         # Calculate gradients for ALL parameters (X, W, b) simultaneously
9 |         grads = tape.gradient(cost_value, [X, W, b])
10 |
11 |         # Update parameters using Adam optimizer
12 |         optimizer.apply_gradients(zip(grads, [X, W, b]))
```

Why Do It This Way?

Since we are implementing a custom cost function that learns the features x (which are variables, not constant inputs), we need the flexibility of `GradientTape`. It allows TensorFlow to track gradients for x just as easily as it does for w and b .

Dataset: MovieLens

In the practice lab, you will use the real-world **MovieLens** dataset to build this system yourself. You will see how the algorithm learns to predict ratings for actual movies.